Le Clean Code

Introduction

Le **Clean Code** (code propre) est une philosophie de développement qui vise à produire du code lisible, maintenable et évolutif. Ce concept a été popularisé par Robert C. Martin (Uncle Bob) dans son ouvrage "Clean Code: A Handbook of Agile Software Craftsmanship".

> *"Un code propre est un code qui a été écrit par quelqu'un qui s'en soucie."*
> *— Robert C. Martin*

---

1. Les Principes Fondamentaux

1.1 La Lisibilité avant tout

Le code est lu beaucoup plus souvent qu'il n'est écrit. Un développeur passe environ **10 fois plus de temps à lire du code qu'à en écrire**.

```
# ❌ Code illisible
def calc(a, b, c):
    return a * b - c / 2 + a

# ✅ Code lisible
def calculer_prix_avec_remise(prix_unitaire, quantite, remise):
    prix_total = prix_unitaire * quantite
    montant_remise = remise / 2
    return prix_total - montant_remise + prix_unitaire
```

1.2 Le Principe de Responsabilité Unique (SRP)

Chaque fonction, classe ou module ne doit avoir qu'**une seule raison de changer**.

```
# ❌ Fonction avec plusieurs responsabilités
def traiter_commande(commande):
    # Valide la commande
    # Calcule le prix
    # Envoie un email
    # Sauvegarde en base
    pass

# ✅ Responsabilités séparées
def valider_commande(commande):
    pass

def calculer_prix_commande(commande):
    pass

def envoyer_confirmation(commande):
    pass

def sauvegarder_commande(commande):
    pass
```

2. Le Nommage

Le nommage est l'un des aspects les plus importants du clean code. Un bon nom révèle l'**intention** et le **contexte**.

2.1 Règles de Nommage

RÈGLE	MAUVAIS EXEMPLE	BON EXEMPLE
Noms révélateurs d'intention	`d`	`elapsed_time_in_days`
Éviter les abréviations	`usr_mgr`	`user_manager`
Noms prononçables	`genymdhms`	`generation_timestamp`
Noms recherchables	`7`	`MAX_STUDENTS_PER_CLASS`
Éviter les préfixes inutiles	`m_name`	`name`

2.2 Conventions par Type

```
# Variables : snake_case, descriptif
nombre_utilisateurs_actifs = 42
prix_total_ttc = 150.00

# Constantes : SCREAMING_SNAKE_CASE
TAUX_TVA = 0.20
MAX_RETRY_ATTEMPTS = 3

# Fonctions : verbe + complément
def calculer_moyenne(notes):
    pass

def est_valide(email):  # Prédicat avec "est_" ou "is_"
    pass

# Classes : PascalCase, nom
class GestionnaireCommandes:
    pass
```

2.3 La Règle du Contexte

Le nom doit être suffisamment explicite dans son contexte :

```
# ❌ Contexte insuffisant
class Adresse:
    def __init__(self):
        self.ville = ""
        self.rue = ""
        self.etat = ""  # État de quoi ? De l'adresse ? Du pays ?

# ✅ Contexte clair
class Adresse:
    def __init__(self):
        self.ville = ""
        self.rue = ""
        self.region = ""  # Clair dans le contexte d'une adresse
```

3. Les Fonctions

3.1 Petites et Focalisées

Une fonction doit être **petite** (idéalement 5-15 lignes) et ne faire qu'**une seule chose**.

```python
# ❌ Fonction trop longue et complexe
def traiter_donnees(donnees):
    resultats = []
    for item in donnees:
        if item.type == "A":
            # 20 lignes de traitement...
            pass
        elif item.type == "B":
            # 20 lignes de traitement...
            pass
        # etc.
    return resultats

# ✅ Fonctions petites et focalisées
def traiter_donnees(donnees):
    return [traiter_item(item) for item in donnees]

def traiter_item(item):
    traitements = {
        "A": traiter_type_a,
        "B": traiter_type_b,
    }
    traitement = traitements.get(item.type, traiter_type_defaut)
    return traitement(item)
```

3.2 Nombre d'Arguments

Limitez le nombre d'arguments. Idéalement **0 à 2**, maximum **3**.

```python
# ❌ Trop d'arguments
def creer_utilisateur(nom, prenom, email, age, ville, pays, telephone, role):
    pass

# ✅ Utiliser un objet
@dataclass
class InfosUtilisateur:
    nom: str
    prenom: str
    email: str
    age: int
    adresse: Adresse
    telephone: str
    role: str

def creer_utilisateur(infos: InfosUtilisateur):
    pass
```

3.3 Pas d'Effets de Bord

Une fonction ne doit pas modifier d'état caché ou produire des effets inattendus.

```python
# ❌ Effet de bord caché
class Authentification:
    def verifier_mot_de_passe(self, utilisateur, mot_de_passe):
        if self._valider(mot_de_passe):
            self.session.initialiser()  # Effet de bord inattendu !
            return True
        return False

# ✅ Fonction pure
class Authentification:
    # verifier
    def verifier_mot_de_passe(self, utilisateur, mot_de_passe):
        return self._valider(mot_de_passe)

    def initialiser_session(self, utilisateur):
        self.session.initialiser()
```

3.4 Command Query Separation (CQS)

Une fonction doit soit **faire quelque chose** (Command), soit **retourner quelque chose** (Query), mais **jamais les deux**.

```
# ❌ Mélange command et query
def set_and_get_age(self, age):
    self.age = age
    return self.age

# ✅ Séparation
def set_age(self, age):
    self.age = age

def get_age(self):
    return self.age
```

---

4. Les Commentaires

4.1 Le Code doit s'Auto-Documenter

Le meilleur commentaire est celui qu'on n'a pas besoin d'écrire.

```
# ❌ Commentaire redondant
# Incrémente le compteur de 1
compteur += 1

# ❌ Commentaire pour compenser un mauvais nommage
# Vérifie si l'utilisateur a plus de 18 ans
if u.a >= 18:
    pass

# ✅ Code auto-documenté
if utilisateur.est_majeur():
    pass
```

4.2 Commentaires Acceptables

```
# Explication d'une décision technique complexe
# Utilisation de l'algorithme de Dijkstra car le graphe est peu dense
# et les poids sont toujours positifs

# Avertissement sur les conséquences
# ATTENTION: Cette fonction invalide le cache global

# TODO pour les tâches futures (à utiliser avec parcimonie)
# TODO: Optimiser cette requête quand le volume dépassera 10k entrées

# Documentation d'API publique
def calculer_distance(point_a: Point, point_b: Point) -> float:
    """
    Calcule la distance euclidienne entre deux points.

    Args:
        point_a: Premier point
        point_b: Second point

    Returns:
        Distance en unités
    """
    pass
```

4.3 Commentaires à Éviter

```
# ❌ Commentaires de fermeture
if condition:
    # beaucoup de code...
    pass
# fin if  <-- Inutile, refactorisez plutôt

# ❌ Code commenté
# ancienne_fonction()
# autre_code_obsolete()

# ❌ Journal des modifications
# Modifié le 12/01/2024 par Jean - ajout validation
# Modifié le 15/01/2024 par Marie - correction bug
# --> Utilisez Git !

# ❌ Commentaires bruyants
# Constructeur par défaut
def __init__(self):
    pass
```

---

5. Le Formatage

5.1 Organisation Verticale

Le code doit se lire comme un **journal** : du plus important/général vers le plus détaillé.

```python
# ✅ Organisation en "journal"

# 1. Imports (du plus général au plus spécifique)
import os
import sys
from typing import List, Optional

from mon_projet.core import Config
from mon_projet.utils import formater

# 2. Constantes
TAILLE_MAX_FICHIER = 1024 * 1024

# 3. Fonctions publiques (API)
def traiter_fichier(chemin: str) -> Resultat:
    contenu = lire_fichier(chemin)
    donnees = parser_contenu(contenu)
    return transformer_donnees(donnees)

# 4. Fonctions privées/utilitaires (détails)
def lire_fichier(chemin: str) -> str:
    pass

def parser_contenu(contenu: str) -> dict:
    pass

def transformer_donnees(donnees: dict) -> Resultat:
    pass
```

5.2 Espacement et Densité

```
# ✅ Bon espacement
class Calculateur:

    def __init__(self, precision: int = 2):
        self.precision = precision

    def additionner(self, a: float, b: float) -> float:
        resultat = a + b
        return round(resultat, self.precision)

    def soustraire(self, a: float, b: float) -> float:
        resultat = a - b
        return round(resultat, self.precision)
```

5.3 Longueur des Lignes

Limitez les lignes à **80-120 caractères**.

```
# ❌ Ligne trop longue
resultat = ma_fonction_avec_un_nom_tres_long(premier_argument_detaille, deuxieme_argum

# ✅ Ligne découpée proprement
resultat = ma_fonction_avec_un_nom_tres_long(
    premier_argument_detaille,
    deuxieme_argument_detaille,
    troisieme_argument_detaille
)
```

6. Gestion des Erreurs

6.1 Exceptions plutôt que Codes de Retour

```
# ❌ Codes de retour
def diviser(a, b):
    if b == 0:
        return None, "Division par zéro"
    return a / b, None

resultat, erreur = diviser(10, 0)
if erreur:
    print(erreur)

# ✅ Exceptions
def diviser(a: float, b: float) -> float:
    if b == 0:
        raise ValueError("Division par zéro impossible")
    return a / b

try:
    resultat = diviser(10, 0)
except ValueError as e:
    logger.error(f"Erreur de calcul: {e}")
```

6.2 Fail Fast

Échouez le plus tôt possible, validez les entrées immédiatement.

```python
# ✅ Validation immédiate (Guard Clauses)
def traiter_commande(commande: Commande) -> Resultat:
    if commande is None:
        raise ValueError("La commande ne peut pas être nulle")

    if not commande.articles:
        raise ValueError("La commande doit contenir au moins un article")

    if commande.montant_total <= 0:
        raise ValueError("Le montant doit être positif")

    # Logique métier (pas imbriquée dans des if)
    return executer_commande(commande)

def nested_if(user_input):
    if user_input.is_valid:
        #Process
        #Process
        #Process
        #Process
        #Process
        #Process
    else :
        return Error


def not_nested_if(user_input):
    if not user_input.is_valid:
        return Error
    #Process
    #Process
    #Process
    #Process
    #Process
    #Process
```

6.3 Ne pas Retourner/Passer null

```
# ❌ Retourner null
def trouver_utilisateur(id: int):
    utilisateur = self.repository.get(id)
    if utilisateur is None:
        return None  # Utilisateur non trouvé
    # ...
    return utilisateur

# ✅ Lever une exception ou retourner un Optional explicite
def trouver_utilisateur(id: int) -> Utilisateur:
    utilisateur = self.repository.get(id)
    if utilisateur is None:
        raise UtilisateurNonTrouveError(f"Utilisateur {id} introuvable")
    return utilisateur

# Ou avec Optional
def trouver_utilisateur(id: int) -> Optional[Utilisateur]:
    return self.repository.get(id)  # Le type explicite force la vérification
```

7. Le Principe DRY (Don't Repeat Yourself)

7.1 Éliminer la Duplication

```
# ❌ Code dupliqué
def calculer_prix_produit_a(quantite):
    prix_base = 10.0
    tva = prix_base * 0.20
    remise = prix_base * 0.05 if quantite > 10 else 0
    return (prix_base + tva - remise) * quantite

def calculer_prix_produit_b(quantite):
    prix_base = 25.0
    tva = prix_base * 0.20
    remise = prix_base * 0.05 if quantite > 10 else 0
    return (prix_base + tva - remise) * quantite

# ✅ Factorisation
def calculer_prix(prix_base: float, quantite: int, seuil_remise: int = 10) -> float:
    tva = prix_base * TAUX_TVA
    remise = prix_base * TAUX_REMISE if quantite > seuil_remise else 0
    return (prix_base + tva - remise) * quantite
```

7.2 Attention à la Fausse Duplication

Ne factorisez pas du code qui se ressemble par hasard mais qui a des raisons différentes de changer.

```python
# ⚠ Ressemble à de la duplication mais...
def valider_email_inscription(email):
    #Validation
    #Process DB
    # Renvoie du state
    # Règles spécifiques à l'inscription
    pass

def valider_email_newsletter(email):
    #Validation
    #Process DB
    # Renvoie du state
    # Règles spécifiques à la newsletter (peuvent diverger)
    pass


def valider_email_genereic(email,validation,process,renvoie):
    validation(email)
    process(email)
    return renvoie(email)

def valider_email_genereic(email,validation,processes,renvoies):
    validation(email)
    for process in processes:
        process(email)
    return all(renvoie(email))

def validate_mail(email):
    return email.find("@")

def enregistrement_DB(mail):
    print("enregistrer en DB")

def enregistrement_newsLetter(mail):
    print("enregistrer sur la NewsLetter")


def enregistrement_FranceConnect(mail):
    print("enregistrer sur France Connect")
```

```
def check_France_Connect(email):
    print("check")
    return True

def check_DB(email):
    print("check")
    return True

def check_NewsLetter(email):
    print("check")
    return True

def news_letter(mail):
    valider_email_genereic(
        mail,
        validate_mail,
        [enregistrement_DB,enregistrement_newsLetter,enregistrement_franceConnect],
        [check_DB,check_NewsLetter,verif]
    )
def main():
    mail = "haha@gmail.com"

    valider_email_genereic(
        mail,
        validate_mail,
        [enregistrement_DB,enregistrement_franceConnect],
        [check_DB,check_FranceConnect]
    )
```

8. Les Principes SOLID

8.1 S - Single Responsibility Principle (SRP)

Une classe ne doit avoir qu'une seule raison de changer.

```
# ❌ Classe avec multiples responsabilités
class Rapport:
    def calculer_donnees(self):
        pass

    def formater_html(self):
        pass

    def envoyer_email(self):
        pass
```

```
# ✅ Responsabilités séparées
class CalculateurRapport:
    def calculer(self, donnees):
        pass

class FormateurRapport:
    def formater_html(self, rapport):
        pass

class EnvoyeurRapport:
    def envoyer(self, rapport, destinataire):
        pass
```

8.2 O - Open/Closed Principle (OCP)

Ouvert à l'extension, fermé à la modification.

```
# ❌ Modification nécessaire pour chaque nouveau type
def calculer_aire(forme):
    if forme.type == "cercle":
        return 3.14 * forme.rayon ** 2
    elif forme.type == "rectangle":
        return forme.largeur * forme.hauteur
    # Chaque nouvelle forme = modification

#JT = {
#    "cercle": (forme)=> 3.14 *,
#    "rectangle": (forme)=> 3.14 *
#    "triangle": (forme)=> 3.14 *
#    "triangle": (forme)=> 3.14 *
#    "triangle": (forme)=> 3.14 *
#    "triangle": (forme)=> 3.14 *
#    "triangle": (forme)=> 3.14 *
#}

#def calc(forme):
#    return JT[forme.type](forme)
```

```python
# ✅ Extension sans modification
from abc import ABC, abstractmethod

class Forme(ABC):
    @abstractmethod
    def calculer_aire(self) -> float:
        pass

class Cercle(Forme):
    def __init__(self, rayon: float):
        self.rayon = rayon

    def calculer_aire(self) -> float:
        return 3.14 * self.rayon ** 2

class Rectangle(Forme):
    def __init__(self, largeur: float, hauteur: float):
        self.largeur = largeur
        self.hauteur = hauteur

    def calculer_aire(self) -> float:
        return self.largeur * self.hauteur
```

8.3 L - Liskov Substitution Principle (LSP)

Les sous-types doivent pouvoir remplacer leurs types de base.

```
# ❌ Violation du LSP
class Oiseau:
    def voler(self):
        pass

class Pingouin(Oiseau):
    def voler(self):
        raise Exception("Les pingouins ne volent pas !")

# ✅ Hiérarchie correcte
class Oiseau:
    def se_deplacer(self):
        pass

class OiseauVolant(Oiseau):
    def voler(self):
        pass

class Pingouin(Oiseau):
    def nager(self):
        pass
```

8.4 I - Interface Segregation Principle (ISP)

Préférez plusieurs interfaces spécifiques à une interface générale.

```python
# ❌ Interface trop large
class Travailleur(ABC):
    @abstractmethod
    def travailler(self):
        pass

    @abstractmethod
    def manger(self):
        pass

    @abstractmethod
    def dormir(self):
        pass

class Robot(Travailleur):
    def manger(self):
        pass  # Ne s'applique pas !

# ✅ Interfaces séparées
class Travailleur(ABC):
    @abstractmethod
    def travailler(self):
        pass

class EtreVivant(ABC):
    @abstractmethod
    def manger(self):
        pass

    @abstractmethod
    def dormir(self):
        pass

class Humain(Travailleur, EtreVivant):
    pass

class Robot(Travailleur):
    pass
```

8.5 D - Dependency Inversion Principle (DIP)

Dépendez des abstractions, pas des implémentations.

```
# ❌ Dépendance directe
class ServiceNotification:
    def __init__(self):
        self.smtp = SMTPClient()  # Couplage fort

    def envoyer(self, message):
        self.smtp.send(message)

# ✅ Injection de dépendance
class EnvoyeurMessage(ABC):
    @abstractmethod
    def envoyer(self, message: str):
        pass

class EnvoyeurEmail(EnvoyeurMessage):
    def envoyer(self, message: str):
        # Implémentation SMTP
        pass

class EnvoyeurSMS(EnvoyeurMessage):
    def envoyer(self, message: str):
        # Implémentation SMS
        pass

class ServiceNotification:
    def __init__(self, envoyeur: EnvoyeurMessage):
        self.envoyeur = envoyeur

    def notifier(self, message: str):
        self.envoyeur.envoyer(message)
```

9. Les Code Smells (Mauvaises Odeurs)

9.1 Signes d'Alerte

CODE SMELL	DESCRIPTION	SOLUTION
Fonction longue	> 20-30 lignes	Extraire en sous-fonctions
Classe géante	Trop de responsabilités	Diviser en classes cohérentes
Liste de paramètres longue	> 3 paramètres	Créer un objet paramètre
Commentaires excessifs	Code peu clair	Refactoriser le code
Code dupliqué	Copier-coller	Extraire et réutiliser
Feature Envy	Méthode utilise trop une autre classe	Déplacer la méthode
Data Clumps	Données toujours ensemble	Créer une classe
Switch statements	Switch/if-else répétés	Polymorphisme

9.2 Exemple de Refactoring

```
# ❌ Avant : Multiple code smells
def traiter(data, type, format, debug, log, notify, cache):
    if type == "A":
        # 50 lignes de code...
        if format == "json":
            # ...
        elif format == "xml":
            # ...
    elif type == "B":
        # 50 lignes similaires...
        pass

    if notify:
        # Envoi notification...
        pass

    if log:
        # Logging...
        pass

# ✅ Après : Code propre
@dataclass
class ConfigTraitement:
    format: str
    debug: bool = False
    log: bool = True
    notify: bool = False
    cache: bool = True

class Processeur(ABC):
    @abstractmethod
    def traiter(self, data, config: ConfigTraitement):
        pass

class ProcesseurTypeA(Processeur):
    def traiter(self, data, config: ConfigTraitement):
        formateur = self._get_formateur(config.format)
        return formateur.formater(data)
```

```
class ProcesseurTypeB(Processeur):
    def traiter(self, data, config: ConfigTraitement):
        pass

class ServiceTraitement:
    def __init__(self, processeurs: dict[str, Processeur]):
        self.processeurs = processeurs

    def traiter(self, data, type: str, config: ConfigTraitement):
        processeur = self.processeurs[type]
        return processeur.traiter(data, config)
```

---

10. Bonnes Pratiques Additionnelles

10.1 Tests et Clean Code

Le code de test doit aussi être propre :

```
# ✅ Test lisible avec pattern AAA (Arrange-Act-Assert)
def test_calcul_remise_pour_client_premium():
    # Arrange
    client = Client(type=TypeClient.PREMIUM)
    commande = Commande(montant=100, client=client)

    # Act
    remise = calculer_remise(commande)

    # Assert
    assert remise == 15  # 15% pour les clients premium
```

10.2 Loi de Déméter

Un objet ne doit parler qu'à ses amis immédiats.

```
# ❌ Train Wreck
client.get_compte().get_solde().get_devise().get_symbole()

# ✅ Tell, Don't Ask
client.obtenir_symbole_devise()
```

10.3 Composition vs Héritage

Préférez la composition à l'héritage quand possible.

```
# ❌ Héritage profond
class Animal:
    pass

class Mammifere(Animal):
    pass

class Chien(Mammifere):
    pass

class ChienDeChasse(Chien):
    pass

# ✅ Composition
class Comportement:
    pass

class ComportementChasse(Comportement):
    def chasser(self):
        pass

class Chien:
    def __init__(self, comportement: Comportement = None):
        self.comportement = comportement
```

11. Checklist du Clean Code

Avant de valider votre code, vérifiez :

- [ ] Les noms sont-ils explicites et révélateurs d'intention ?
- [ ] Les fonctions sont-elles courtes et font-elles une seule chose ?
- [ ] Le code est-il auto-documenté (commentaires minimaux) ?
- [ ] Y a-t-il du code dupliqué à factoriser ?
- [ ] Les erreurs sont-elles gérées proprement ?
- [ ] Le formatage est-il cohérent ?
- [ ] Les dépendances sont-elles correctement injectées ?
- [ ] Les tests sont-ils lisibles et maintenables ?
- [ ] Le code respecte-t-il les principes SOLID ?
- [ ] Y a-t-il des code smells à corriger ?

Conclusion

Le Clean Code n'est pas une destination, c'est un **voyage continu**. Il demande :

- **Discipline** : Appliquer les principes même sous pression
- **Pratique** : Refactoriser régulièrement
- **Humilité** : Accepter que son code peut toujours être amélioré
- **Collaboration** : Faire des revues de code

> *"Laissez le code plus propre que vous ne l'avez trouvé."*
> *— La règle du Boy Scout*

Le temps investi dans le clean code est rentabilisé par :

- Une maintenance plus facile
- Moins de bugs
- Une équipe plus efficace
- Un produit plus fiable

Ressources Complémentaires

- **Clean Code -** Robert C. Martin
- **The Pragmatic Programmer -** David Thomas, Andrew Hunt
- **Refactoring -** Martin Fowler
- **Design Patterns -** Gang of Four